El Project Manager en la Era de la IA: De Administrador a Estratega

Micaela Alvarado Mendez
Universidad Nacional de Tierra del Fuego
Ushuaia, Argentina
micaelaalvaradomendez@gmail.com

***Resumen*—Históricamente, la gestión de proyectos en desarrollo de software ha estado absorbida por una pesada carga administrativa orientada a la recolección, transcripción y reporte de tareas. La integración de la Inteligencia Artificial (IA) y los modelos de lenguaje a gran escala permite automatizar estas actividades operativas —como la generación de minutas, documentación y seguimiento de avances—, liberando recursos cognitivos esenciales del Project Manager (PM). Este artículo analiza la transición de dicho rol tradicional hacia un perfil de habilitador estratégico y AI Manager. Lejos de prescindir del factor humano, la abundancia de datos algorítmicos traslada el verdadero desafío hacia la resolución de la brecha de juicio (judgment gap). A través de la ingeniería de contexto, la supervisión human-in-the-loop y el desarrollo de habilidades interpersonales —como el sentido crítico, el coraje estratégico y la generación de confianza—, se redefine un liderazgo enfocado en la toma de decisiones críticas dentro de sistemas sociotécnicos de software.**

***Index Terms*—Inteligencia Artificial, Project Management, Ingeniería de Software, Habilitador Estratégico, Automatización, Toma de Decisiones Críticas, Judgment Gap, AI Manager**

I. Introducción

Durante décadas, la gestión de proyectos de software se organizó alrededor de una premisa incuestionada: obtener información era costoso. Los datos sobre el avance de un equipo se recolectaban manualmente, los reportes tardaban semanas en consolidarse y las estimaciones dependían de la experiencia acumulada de unos pocos. Bajo este paradigma, el Project Manager (PM) asumió un rol predominantemente administrativo y burocrático [1]: redactar y desglosar historias de usuario en tableros de seguimiento, tomar minutas de reuniones, armar reportes de estado para los interesados (*stakeholders*), corregir la redacción de correos técnicos y calcular estimaciones de forma empírica [1], [2]. La gestión se convirtió, en la práctica, en una tarea de transcripción y control.

Esta concentración excesiva en el papeleo genera dos consecuencias críticas. Primero, produce cuellos de botella: el PM se convierte en un cuello de botella humano que ralentiza la circulación de información entre equipos. Segundo, y más grave, aparta la vista de lo fundamental. Un proyecto de software no es una lista de tareas que se van tildando, sino un entorno dinámico atravesado por decisiones de arquitectura, riesgos de seguridad en el código, cambios de requerimientos y desafíos de coordinación entre personas [1], [3]. Cuando el líder queda atrapado en la administración rutinaria, pierde capacidad para anticipar riesgos en el camino crítico, frenar el avance de la deuda técnica y sostener la moral del equipo en momentos de presión.

Las investigaciones recientes confirman esta disociación. Barbosa et al. (2025), a partir de quince entrevistas en profundidad con PMs de distintos países, sectores y tamaños organizacionales, documentan que las tareas administrativas resumir documentación extensa, estructurar correos, corregir gramática y redactar minutas consumen una porción significativa de la jornada del gestor, sin aportar valor técnico directo al producto [2]. Irónicamente, estas son precisamente las tareas que hoy pueden automatizarse con mayor facilidad.

La irrupción de la Inteligencia Artificial (IA) generativa, y en particular de los modelos de lenguaje a gran escala (*Large Language Models*, LLM), modifica drásticamente este panorama [2]. Hoy, estos modelos son capaces de resumir reuniones técnicas, generar borradores de requerimientos, proponer distribuciones de trabajo, verificar contratos de API y detectar patrones de riesgo en repositorios de código, todo en segundos. Los datos recogidos por Barbosa et al. (2025) son elocuentes: uno de sus entrevistados estima ganancias de eficiencia cercanas al 70 % y reducciones de costos del 30 % en tareas administrativas [2]. La promesa es tentadora: liberar al PM de la carga operativa para que se concentre en lo estratégico.

Sin embargo, como advertía Herbert Simon ya en 1978, *"una riqueza de información crea una pobreza de atención"* [3]. Las herramientas automáticas resuelven el cálculo analítico y el armado de reportes, pero abren una dificultad mayor: la brecha de juicio (*judgment gap*), es decir, la distancia entre ver un panel lleno de gráficos predictivos y entender qué es lo que realmente conviene hacer con ellos [3]. Un tablero puede recomendar recortar funcionalidades o reasignar módulos, pero no asume las consecuencias si el cliente queda disconforme o si el producto falla en producción. Las consecuencias siempre las asumen las personas [3].

Esta distinción es crucial y suele pasarse por alto en la literatura entusiasta sobre IA. La tecnología cierra la brecha analítica (*decision gap*) procesar escenarios con rapidez , pero acentúa la brecha de juicio [3]. En la era de la información abundante, el valor del gestor ya no radica en conseguir datos, sino en saber filtrarlos y descartar el ruido [3].

En este nuevo contexto, el gestor deja de ser un mero registrador de tareas y evoluciona hacia dos figuras complementarias: el habilitador estratégico (*strategic enabler*) [2] y el

AI Manager [1]. Este perfil no necesita programar algoritmos, sino dominar la ingeniería de contexto (*context engineering*) para alimentar a los asistentes de IA con las restricciones del sistema, los contratos de interfaz (como esquemas OpenAPI), las reglas del negocio y la dinámica del equipo, manteniendo una supervisión humana constante (*human-in-the-loop*) que evite errores conceptuales [1], [2].

La metáfora que ofrece uno de los entrevistados por Barbosa et al. (2025) es particularmente iluminadora:

> *"El prompt engineering es como darle una tarea a un minion; el contexto es usarlo como copiloto."* (R7)

Mientras que un prompt aislado trata a la IA como un asistente mecánico, el diseño de contexto la convierte en un verdadero copiloto de trabajo [2]. Al contar con el panorama completo, la herramienta genera historias de usuario coherentes, planes de prueba aplicables y especificaciones que respetan los objetivos reales del software [1], [2].

Lejos de reemplazar a las personas, la automatización pone en primer plano las cualidades puramente humanas [3]. Como los algoritmos pueden sugerir escenarios pero nunca hacerse cargo de sus consecuencias, liderar proyectos de software exige:

- Criterio contextual (*sense-making*): distinguir qué señales importan y cuáles son simple ruido [3].
- Coraje estratégico (*strategic courage*): elegir qué priorizar ante la incertidumbre, cuando la información nunca es 100 % segura [3].
- Construcción de confianza (*trust creation*): desarrollar acuerdos genuinos y credibilidad compartida entre desarrolladores y directivos [3].

Ninguna de estas tres capacidades puede automatizarse fácilmente, porque dependen del contexto, la cultura organizacional y el juicio moral [3].

Un aspecto que la literatura sobre IA en gestión de proyectos ha comenzado a documentar y que este artículo busca destacar es la asimetría en la adopción según la experiencia del gestor. Barbosa et al. (2025) encontraron que los PMs seniors tienden a ser más precavidos y escépticos: priorizan métodos tradicionales probados y prestan mayor atención a la seguridad y la gobernanza. Por el contrario, los juniors adoptan la IA con naturalidad y confían rápidamente en sus respuestas [2].

Esta situación genera una paradoja que ninguna herramienta resuelve por sí sola: los gestores noveles manejan las herramientas con soltura, pero muchas veces carecen del criterio técnico para detectar errores sutiles o alucinaciones en el diseño; en cambio, los profesionales experimentados tienen el juicio necesario para validar los entregables, pero les cuesta más incorporar la IA en su rutina diaria [2], [3]. Integrar ambas miradas resulta indispensable para aprovechar la velocidad de la tecnología sin comprometer la solidez técnica del sistema [1].

El presente trabajo examina esta transformación del rol del PM en la era de la IA a través de cuatro etapas:

1. El paso de las tareas operativas a la ingeniería de contexto [2]: cómo la automatización de la administración libera recursos cognitivos y traslada el valor del gestor hacia el diseño de contexto.
2. Las competencias del AI Manager a lo largo del ciclo de vida del software [1]: un modelo tripartito (técnico, cognitivo, ético) que operacionaliza las capacidades necesarias para liderar proyectos mediados por IA.
3. La superación del *judgment gap* a través del liderazgo humano [3]: por qué el juicio, el coraje y la confianza se vuelven más valiosos no menos en un entorno de información abundante.
4. La gestión de cuellos de botella y la verificación de código asistida por IA: cómo la supervisión técnica se transforma cuando los agentes automatizados auditan *diffs*, validan contratos de API y detectan deuda técnica antes de que se consolide.

A lo largo del artículo se incorporan tablas comparativas y figuras que sintetizan los hallazgos de las fuentes consultadas y proponen un marco operativo para el AI Manager. Finalmente, se plantean las tensiones y brechas que persisten entre eficiencia y criterio, entre automatización y autoría, entre marcos normativos globales y capacidades locales en América Latina , con el objetivo de ofrecer una mirada crítica y no meramente descriptiva de esta transformación.

II. De la tarea administrativa a la ingeniería de contexto

La primera etapa de la transformación del rol del PM no consiste en *usar* IA, sino en **redefinir qué tareas merecen su atención**. La literatura reciente coincide en un diagnóstico: la gestión tradicional de proyectos de software está cargada de fricciones operativas que consumen tiempo sin aportar valor técnico directo [1], [2]. La respuesta intuitiva —automatizar esas tareas con modelos de lenguaje— es correcta pero incompleta: como se argumenta en esta sección, **el valor del gestor no se traslada a la herramienta, sino al diseño de las condiciones en las que la herramienta opera**. Ese desplazamiento es lo que la literatura denomina *context engineering* y constituye la habilidad central del perfil emergente.

A. El costo invisible de la gestión tradicional

La carga administrativa del PM no es un problema menor ni un simple defecto de eficiencia: es un problema **estructural**. Actividades como redactar actas, mantener al día los tickets del *backlog*, confeccionar informes de estado y corregir la redacción de correos técnicos deben realizarse, pero no agregan valor técnico al producto [1], [3]. El resultado es una paradoja operativa: el gestor dedica la mayor parte de su jornada a tareas que, si desaparecieran, el proyecto no perdería calidad técnica.

Barbosa et al. [2], a partir de quince entrevistas en profundidad con PMs de distintos países, sectores y tamaños organizacionales, documentan que estas tareas —resumir documentación extensa, estructurar correos, corregir gramática

Cuadro I: Análisis de fricción operativa en la gestión tradicional de proyectos

Actividad	Frecuencia	Tiempo	Valor técnico	Automatizable
Redacción de minutas	Diaria/semanal	Alto	Nulo	Sí
Actualización de tickets	Diaria	Medio	Bajo	Parcial
Reportes de estado	Semanal	Alto	Bajo	Sí
Corrección de redacción	Diaria	Bajo	Nulo	Sí
Desglose de historias de usuario	Por sprint	Alto	Medio	Parcial
Estimaciones empíricas	Por sprint	Alto	Medio	Parcial
Análisis de riesgos	Continuo	Bajo	Alto	No
Coordinación entre equipos	Diaria	Medio	Alto	No
Negociación con *stakeholders*	Semanal	Medio	Alto	No

y redactar minutas— ocupan una porción significativa de la jornada del gestor. La Tabla I sintetiza el análisis de fricción operativa que se desprende de estos hallazgos.

La tabla revela un patrón incómodo: las tareas que más tiempo consumen (minutas, reportes, desglose de historias) son precisamente las que **menos valor técnico aportan** y las que **mayor potencial de automatización** presentan. En cambio, las tareas que sí aportan valor —análisis de riesgos, coordinación, negociación— ocupan menos tiempo del que deberían, precisamente porque el gestor está atrapado en las primeras.

Esta concentración excesiva en el papeleo genera dos consecuencias críticas. Primero, produce **cuellos de botella**: el PM se convierte en un intermediario humano que ralentiza la circulación de información entre equipos [1]. Segundo, y más grave, **aparta la vista de lo fundamental**: un proyecto de software no es una lista de tareas que se van tildando, sino un entorno dinámico atravesado por decisiones de arquitectura, riesgos de seguridad en el código, cambios de requerimientos y desafíos de coordinación entre personas [1], [3]. Cuando el líder queda atrapado en la administración rutinaria, pierde capacidad para anticipar riesgos en el camino crítico, frenar el avance de la deuda técnica y sostener la moral del equipo en momentos de presión [4].

B. Automatización operativa: qué se gana y qué no

La incorporación de modelos de lenguaje reduce esta carga burocrática de forma directa [2]. En la práctica diaria, los gestores de software utilizan la IA principalmente para tres funciones:

1. **Síntesis de documentación extensa**: resumir reuniones técnicas, consolidar actas y extraer puntos clave de transcripciones.
2. **Estructuración de comunicaciones**: redactar correos técnicos, estandarizar el tono y adaptar el mensaje a audiencias diversas (desarrolladores, directivos, clientes).
3. **Corrección y normalización**: mejorar la redacción en equipos distribuidos o internacionales, reduciendo malentendidos entre áreas [2].

Los datos recogidos por Barbosa et al. [2] son elocuentes: uno de sus entrevistados estima ganancias de eficiencia cercanas al **70 %** y reducciones de costos del **30 %** en tareas administrativas. Estas funciones agilizan la comunicación interna, estandarizan la forma en que se describen los requerimientos y reducen malentendidos entre áreas de desarrollo [2]. Al delegar la redacción y síntesis mecánica en asistentes inteligentes, el equipo ahorra tiempo y costos, lo que permite que el PM se concentre en planificar la arquitectura de las entregas y asegurar la calidad del producto final [1], [2].

Sin embargo, **la automatización por sí sola no resuelve el problema**. Liberar tiempo es una condición necesaria pero no suficiente: si el PM no sabe qué hacer con ese tiempo liberado, la ganancia se diluye en nuevas tareas operativas o en reuniones sin propósito. La eficiencia administrativa, en otras palabras, **solo se convierte en valor estratégico cuando se traduce en mejores decisiones**. Y para eso hace falta una habilidad que no se automatiza: el diseño de contexto.

C. De prompt engineering a context engineering

A pesar de la utilidad de los modelos de lenguaje, la experiencia demuestra que interactuar mediante instrucciones aisladas (*prompt engineering*) resulta insuficiente para coordinar el desarrollo de un sistema [2]. Una orden simple puede redactar una tarea básica, pero pierde efectividad si la herramienta desconoce el contexto real del proyecto. Como sintetiza uno de los entrevistados por Barbosa et al.:

> *"El prompt engineering es como darle una tarea a un minion; el contexto es usarlo como copiloto."* (R7) [2]

Por esta razón, la habilidad central del gestor actual es la **ingeniería de contexto** (*context engineering*) [2]. Esto consiste en alimentar al asistente de IA con **información estructural**: las reglas del negocio, los contratos de interfaz (como esquemas OpenAPI), las limitaciones de arquitectura, los requerimientos no funcionales y la dinámica del equipo [2], [4]. Mientras que un prompt aislado trata a la IA como a un asistente mecánico, el diseño de contexto la convierte en un verdadero copiloto de trabajo [2]. Al contar con el panorama completo, la herramienta genera historias de usuario coherentes, planes de prueba aplicables y especificaciones que respetan los objetivos reales del software [1], [2].

La Tabla II contrasta ambas prácticas, y la Figura 1 ilustra el flujo de información en cada caso.

C0a. Ejemplo trabajado: una historia de usuario para un endpoint de pago.: La diferencia entre ambas prácticas se aprecia mejor con un caso concreto. Supóngase que el PM necesita generar una historia de usuario para un *endpoint* de pago.

Enfoque de prompt aislado:

> *"Generá una historia de usuario para un endpoint de pago."*

Cuadro II: Prompt engineering vs. context engineering

Dimensión	Prompt engineering	Context engineering
Definición	Instrucción aislada que describe una tarea	Instrucción enriquecida con información estructural del proyecto
Entrada	Texto breve, genérico	Reglas de negocio, contratos API, restricciones, dinámica del equipo
Output típico	Genérico, requiere reescritura	Aplicable, alineado con el sistema
Metáfora	"Darle una tarea a un minion"	"Usarlo como copiloto"
Riesgo principal	Respuestas desconectadas del sistema	Exceso de contexto irrelevante
Habilidad requerida	Redacción	Traducción de conocimiento organizacional a especificaciones

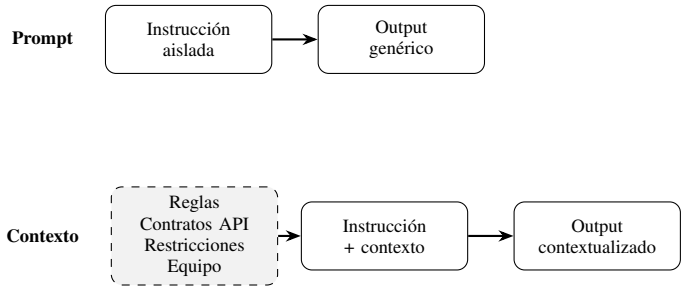


Figura 1: Del prompt engineering al context engineering: la diferencia está en la información estructural que alimenta al modelo.

El output será genérico: una historia que menciona "como usuario quiero pagar para completar mi compra", sin criterios de aceptación específicos, sin restricciones de seguridad y sin alineación con la arquitectura existente. El PM deberá reescribirla casi por completo.

Enfoque de context engineering:

> *"Generá una historia de usuario para un endpoint de pago que cumpla con OpenAPI 3.0, use autenticación JWT, maneje errores 401/403/500, respete la regla de negocio de que ningún pago supere USD 10.000 sin aprobación manual, y se integre con el módulo de auditoría existente documentado en* `/specs/audit.yaml`*."*

El output será directamente aplicable: incluirá criterios de aceptación verificables, casos de error explícitos y referencias a los contratos de interfaz existentes. El PM no reescribe: **valida y ajusta**.

La diferencia no es de herramienta sino de **conocimiento inyectado**. El valor del gestor se desplaza desde la ejecución de la tarea hacia el diseño de las condiciones en las que la tarea se ejecuta. En términos de López Hurtado [1], el AI Manager se convierte en un *traductor de intenciones*: alguien capaz de convertir el conocimiento organizacional disperso —reglas, restricciones, historia del proyecto— en especificaciones que la IA puede procesar.

Cuadro III: Matriz de riesgos técnicos del uso de IA en gestión de proyectos

Riesgo	Descripción	Mitigación
Alucinaciones	Afirmaciones falsas presentadas con seguridad técnica	Validación humana, verificación cruzada
Sesgos	Predicciones que refuerzan desigualdades	Auditoría algorítmica, datos diversos
Filtración de datos	Exposición de información confidencial	Políticas de privacidad, anonimización
Exceso de contexto	Inclusión de información irrelevante que degrada el output	Curaduría de contexto, priorización

Cuadro IV: Asimetría en la adopción de IA según la experiencia del gestor

Dimensión	Senior	Junior
Actitud	Cautelosa, escéptica	Abierta, confiada
Fortaleza	Juicio técnico, experiencia	Soltura tecnológica, adaptabilidad
Debilidad	Menor familiaridad con IA	Menor criterio para detectar errores
Riesgo	Quedarse fuera de la transformación	Confiar ciegamente en los *outputs*

D. Riesgos técnicos y asimetría en la experiencia

El uso de estas soluciones introduce riesgos que exigen un control humano estricto (*human-in-the-loop*) [1]. Los modelos pueden presentar **alucinaciones** —afirmaciones falsas presentadas con total seguridad técnica—, **sesgos** en sus sugerencias o **filtraciones involuntarias** de datos confidenciales si no se definen políticas claras [1], [2]. La Tabla III sintetiza los principales riesgos y sus mitigaciones.

Asimismo, las investigaciones evidencian una marcada diferencia según la experiencia de los gestores [2]. Los perfiles con más años de trayectoria (*seniors*) tienden a ser más precavidos y escépticos: priorizan métodos tradicionales probados y prestan mayor atención a la seguridad y la gobernanza. Por el contrario, los profesionales noveles (*juniors*) adoptan la IA con naturalidad y confían rápidamente en sus respuestas [2]. La Tabla IV sintetiza esta asimetría.

Esta situación genera una **paradoja** que ninguna herramienta resuelve por sí sola: los gestores noveles manejan las herramientas con soltura, pero muchas veces carecen del criterio técnico para detectar errores sutiles o alucinaciones en el diseño; en cambio, los profesionales experimentados tienen el juicio necesario para validar los entregables, pero les cuesta más incorporar la IA en su rutina diaria [2], [3]. **Integrar ambas miradas resulta indispensable** para aprovechar la velocidad de la tecnología sin comprometer la solidez técnica del sistema [1].

La consecuencia práctica es que el *human-in-the-loop* no debe entenderse como un requisito burocrático, sino como una necesidad epistémica: el sistema sociotécnico funciona porque combina la velocidad algorítmica con el juicio humano que

la valida. Sin esa combinación, la eficiencia se convierte en riesgo; con ella, se convierte en capacidad estratégica.

III. El paradigma del AI Manager en el ciclo de vida de software

El impacto de la inteligencia artificial va más allá de acelerar tareas puntuales; transforma la forma de concebir el liderazgo técnico mediante el rol del **AI Manager** [1]. Esta figura no elimina las responsabilidades fundamentales de la gestión, sino que las **complementa**, funcionando como un puente entre las posibilidades del software inteligente y las metas estratégicas de la organización [1], [3]. La diferencia con el PM tradicional no es de herramientas, sino de **lógica de acción**: mientras el primero administra recursos y controla desviaciones, el segundo diseña las condiciones para que los sistemas inteligentes amplifiquen el juicio humano sin sustituirlo.

Esta sección desarrolla el paradigma en tres movimientos. Primero, operacionaliza el modelo de competencias del AI Manager, desagregando cada dimensión en indicadores observables y prácticas concretas. Segundo, muestra cómo la IA reconfigura el ciclo de vida del software, transformando la planificación estática en control adaptativo. Tercero, aborda la gobernanza algorítmica como condición de posibilidad de un liderazgo responsable.

A. Modelo de competencias del AI Manager

El perfil gerencial del AI Manager integra tres dimensiones prácticas de trabajo que no operan de manera jerárquica sino **complementaria** [1]. Un gestor que domine la dimensión técnica pero carezca de criterio ético será eficiente pero irresponsable; uno que tenga juicio ético pero no comprenda los límites de los modelos será prudente pero impotente. El valor del perfil emerge de la **intersección** de las tres dimensiones, no de la suma.

La Tabla V operacionaliza el modelo, desagregando cada dimensión en competencias específicas, indicadores observables y prácticas concretas. Esta operacionalización es necesaria porque la literatura suele quedarse en la enumeración abstracta de las dimensiones, sin responder a la pregunta práctica: *¿cómo se evalúa si un gestor es competente en cada dimensión?*

A0a. Dimensión técnica: comprender sin programar.: La dimensión técnica implica entender cómo operan las tecnologías de IA **sin necesidad de programarlas** [1]. El gestor debe comprender el ciclo de vida del dato, cómo se validan los modelos, qué indican las métricas de precisión y cómo identificar sesgos en la información [1]. Esto le permite traducir problemas de negocio en requisitos comprensibles para especialistas y asistentes digitales [1], [2]. La competencia clave no es la programación, sino la **traducción**: convertir el conocimiento organizacional disperso en especificaciones que la IA puede procesar.

A0b. Dimensión cognitiva: pensar en probabilidades, no en certezas.: La dimensión cognitiva se enfoca en el pensamiento sistémico y probabilístico [1]. El líder entiende que las estimaciones y sugerencias de la IA no son verdades definitivas, sino **proyecciones con margen de error** [1]. Además, mantiene una actitud de aprendizaje constante ante la rápida evolución de los entornos de software [1], [2]. Esta dimensión es la que conecta con el *judgment gap*: el gestor que confunde probabilidad con certeza delega en el algoritmo una decisión que requiere juicio humano.

A0c. Dimensión ética: el núcleo de la responsabilidad gerencial.: La dimensión ética constituye el **núcleo de la responsabilidad gerencial** [1]. El gestor cuida la privacidad de los datos, vela por la transparencia de las decisiones automatizadas y asegura que la tecnología no reproduzca discriminaciones ni perjudique al equipo [1]. El criterio humano y el bienestar común siempre deben estar por encima de la pura conveniencia técnica [1], [3]. Sin esta dimensión, las otras dos se convierten en herramientas de eficiencia sin dirección.

B. La reconfiguración del ciclo de vida: de la planificación estática al control adaptativo

En la gestión clásica, planificar y controlar dependía de estimaciones fijas e informes que miraban lo que ya había ocurrido [1], [4]. Con la asistencia predictiva de la IA, estos procesos se vuelven **continuos y adaptativos** [1]. La Figura 2 ilustra los puntos de intervención de la IA a lo largo del ciclo de vida del software, y la Tabla VI contrasta las prácticas tradicionales con las aumentadas por IA en cada fase.

B0a. Planificación aumentada.: En la etapa de inicio, los modelos analizan el historial de proyectos previos y proponen escenarios sobre el ritmo del equipo, la distribución de cargas y los riesgos potenciales [1], [2]. Durante la planificación, los sistemas generan múltiples escenarios y sugieren combinaciones óptimas de recursos, lo que permite construir **planes adaptativos** que se ajustan dinámicamente según el comportamiento real del entorno [1]. La planificación deja de ser un documento estático para convertirse en un **proceso vivo**, sustentado en datos que se actualizan en tiempo real. El rol del AI Manager aquí no es aceptar ciegamente la recomendación del modelo, sino **validar los supuestos** y comunicar los márgenes de error a los *stakeholders*.

B0b. Ejecución inteligente y coordinación híbrida.: Durante el desarrollo, los sistemas de monitoreo pueden anticipar trabas en la integración o desvíos en los plazos [2]. La IA actúa como **sensor preventivo**: detecta cuellos de botella antes de que se conviertan en problemas críticos. El AI Manager actúa como mediador entre los equipos humanos y las plataformas digitales, garantizando que la colaboración se mantenga fluida, inclusiva y orientada a resultados [1]. La coordinación deja de ser una tarea de seguimiento manual para convertirse en una **negociación fundamentada**: el gestor negocia prioridades con datos concretos, no con percepciones.

B0c. Control adaptativo y aprendizaje organizacional.: El control deja de ser una auditoría sancionatoria y pasa a ser un **seguimiento en tiempo real** que permite corregir rumbos antes de que los problemas afecten la entrega [1]. A diferencia de los informes tradicionales, que entregan una visión retrospectiva, las herramientas basadas en IA generan tableros predictivos capaces de anticipar desviaciones y recomendar acciones correctivas [1]. Este tipo de control inteligente fortalece

Cuadro V: Modelo de competencias del AI Manager: operacionalización

Dimensión	Competencia	Indicador observable	Herramienta / Práctica
Técnica	Alfabetización algorítmica	Interpreta métricas de precisión, detecta sesgos en los datos, distingue correlación de causalidad	Dashboards de validación, análisis de métricas de error
	Ingeniería de contexto	Diseña prompts con reglas de negocio, contratos API y restricciones arquitectónicas	OpenAPI/Swagger, documentación estructurada, especificaciones
	Traducción de necesidades	Convierte problemas de gestión en requisitos comprensibles para especialistas y asistentes digitales	User stories técnicas, criterios de aceptación verificables
Cognitiva	Pensamiento sistémico	Anticipa impactos cruzados entre módulos, identifica dependencias ocultas	Mapas de dependencias, análisis de camino crítico
	Pensamiento probabilístico	Distingue estimación de certeza, comunica márgenes de error a los stakeholders	Análisis de escenarios, intervalos de confianza
	Aprendizaje continuo	Revisa fuentes actualizadas, participa en comunidades, transfiere conocimiento al equipo	Comunidades de práctica, documentación viva
Ética	Gobernanza algorítmica	Documenta supuestos y márgenes de error, establece protocolos de auditoría	AI RMF (NIST), EU AI Act, protocolos de trazabilidad
	Supervisión *human-in-the-loop*	Revisa *outputs* antes de decisiones críticas, detecta alucinaciones	Protocolos de revisión, verificación cruzada
	Responsabilidad social	Vela por la privacidad, evita discriminación algorítmica, asegura transparencia	Auditorías de impacto, políticas de privacidad

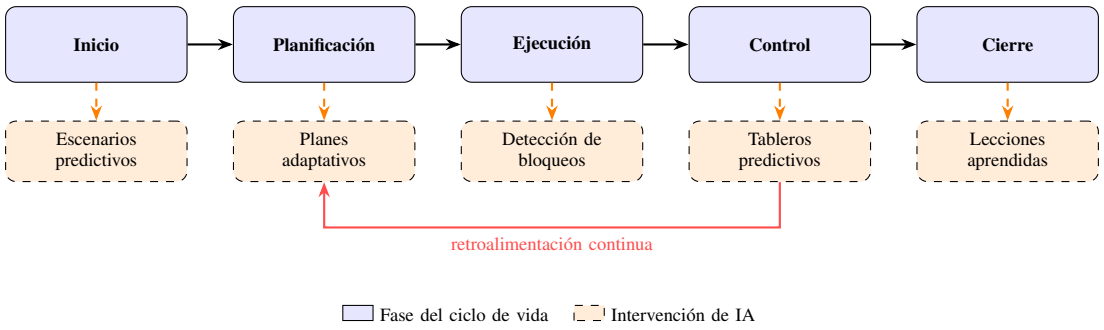


Figura 2: Puntos de intervención de la IA a lo largo del ciclo de vida del software. La retroalimentación continua convierte el control en un proceso adaptativo.

Cuadro VI: Reconfiguración del ciclo de vida: gestión tradicional vs. gestión aumentada por IA

Fase	Gestión tradicional	Gestión aumentada por IA	Rol del AI Manager
Inicio	Estimaciones basadas en experiencia; análisis manual de proyectos previos	Análisis de historial; generación de escenarios de ritmo, carga y riesgo	Selecciona escenarios plausibles; descarta los inviables
Planificación	Plan estático; cronograma fijo; asignación manual de recursos	Planes adaptativos; simulación de combinaciones de recursos; ajuste dinámico	Valida supuestos; comunica márgenes de error
Ejecución	Coordinación manual; detección tardía de bloqueos	Monitoreo de PRs, integración y fallos; alertas tempranas de cuellos de botella	Destraba dependencias; negocia prioridades con fundamento
Control	Informes retrospectivos; auditoría sancionatoria	Tableros predictivos; seguimiento en tiempo real; recomendaciones correctivas	Interpreta señales; decide qué corregir y qué ignorar
Cierre	Lecciones aprendidas documentadas manualmente	Extracción automática de patrones; alimentación de modelos futuros	Traduce datos en conocimiento organizacional

la **cultura del aprendizaje organizacional**: cada iteración del proyecto produce nuevos datos que alimentan los modelos, generando un ciclo de mejora permanente [1]. La PMO se transforma en un espacio de inteligencia adaptativa, donde los errores se interpretan como oportunidades de aprendizaje más que como fallas a sancionar.

Cuadro VII: Marcos de gobernanza algorítmica y sus implicaciones para el AI Manager

Marco	Pilares	Implicación para el AI Manager
EU AI Act (2024)	Sistemas de alto riesgo con auditoría y control humano significativo	Articular políticas de revisión y trazabilidad; garantizar explicabilidad
NIST AI RMF (2023)	Gobernanza, validación, gestión del riesgo, supervisión humana	Documentar supuestos; comunicar márgenes de error; preservar privacidad

C. *Gobernanza algorítmica y supervisión humana*

La utilización seria de la IA en software requiere **reglas claras de supervisión** [1]. Siguiendo estándares internacionales como el EU AI Act o las guías del NIST, las empresas deben asegurar que cada decisión crítica pueda ser **explicada y auditada** [1]. La Tabla VII sintetiza los pilares de ambos marcos y sus implicaciones para el AI Manager.

Bajo el esquema del AI Manager, decisiones vitales como la **aprobación de arquitecturas de seguridad**, los **compromisos de entrega** o la **priorización del producto** jamás se delegan a modelos opacos tipo caja negra [1], [3]. La tecnología asiste y proyecta, pero la **responsabilidad y el gobierno del software permanecen en manos del equipo humano** [1]. El AI Manager se convierte así en el **garante ético** del sistema: no solo dirige equipos, sino que traduce algoritmos en decisiones comprensibles y moralmente justificadas [1].

Esta función de garante no es un añadido burocrático, sino una **condición de posibilidad** del liderazgo aumentado. Sin gobernanza algorítmica, la eficiencia se convierte en riesgo; con ella, se convierte en capacidad estratégica. Como sintetiza López Hurtado [1], el AI Manager encarna un nuevo paradigma de dirección: un liderazgo híbrido que combina inteligencia técnica con sensibilidad humana, orientado al desarrollo sostenible y la equidad.

IV. Superación del *judgment gap*: liderazgo humano en sistemas sociotécnicos

A pesar de que las herramientas actuales generan opciones y sugerencias en instantes, **tomar decisiones en las organizaciones no se ha vuelto más simple** [3]. De hecho, el exceso de reportes y opciones automáticas puede paralizar a los equipos si no hay un criterio claro para discernir qué caminos tomar [3]. Esta paradoja —más información, más opciones, pero no necesariamente mejores decisiones— constituye el núcleo del desafío que esta sección aborda.

La tesis que se sostiene es la siguiente: **la IA cierra la brecha analítica pero acentúa la brecha de juicio**. La primera se refiere a la distancia entre reconocer que se necesita una decisión y contar con el análisis para prepararla; la segunda, a la distancia entre tener el análisis y saber qué hacer con él [3]. Mientras la IA reduce la primera, la segunda permanece —y se amplifica— porque la abundancia de opciones no resuelve el problema de la **priorización bajo incertidumbre**. Superar esa brecha exige tres capacidades que ningún modelo puede automatizar: *sense-making*, coraje estratégico y construcción de confianza.

A. *La paradoja de Simon y la brecha de juicio*

Este escenario confirma la premisa de Herbert Simon: **cuando la información abunda, la atención humana se convierte en el recurso más escaso** [3]. En el pasado, el valor del gestor radicaba en conseguir y procesar datos; hoy, el valor está en **saber filtrarlos y descartar el ruido** [3]. La Figura 3 ilustra este desplazamiento: la IA cubre el trayecto desde la información hasta las opciones, pero el tramo final —de las opciones a la decisión y de la decisión a las consecuencias— permanece en manos humanas.

Aunque la IA ayuda a cerrar la **brecha analítica** (*decision gap*) procesando escenarios con rapidez, **acentúa la brecha de juicio** (*judgment gap*) [3]. Esta brecha es la distancia que existe entre ver un panel lleno de gráficos predictivos y entender qué es lo que realmente conviene hacer con ellos [3]. Un tablero puede recomendar recortar funcionalidades o reasignar módulos, pero no asume las consecuencias si el cliente queda disconforme o si el producto falla en producción; **las consecuencias siempre las asumen las personas** [3].

A0a. Ejemplo trabajado: el dashboard que no responde la pregunta correcta.: Nieto-Rodríguez [3] relata una escena que condensa el problema. Durante un taller con líderes de transformación, un ejecutivo presentó un tablero de portafolio potenciado por IA: el sistema generaba pronósticos predictivos, identificaba riesgos emergentes, resumía la salud de los proyectos y recomendaba acciones correctivas. La tecnología era extraordinaria. Después de la demostración, el autor hizo una pregunta simple:

"¿Qué proyecto detendrías mañana?"

La sala quedó en silencio.

El tablero había resuelto el problema del *reporting*. **No había resuelto el problema del juicio**. La IA podía decir qué proyectos estaban en riesgo, pero no podía decidir cuál sacrificar cuando todos los indicadores eran aceptables pero los recursos eran finitos. Esa decisión —que implica consecuencias políticas, económicas y humanas— sigue siendo humana [3].

B. *El software como ecosistema social*

Un error habitual es considerar a los proyectos de software como problemas puramente matemáticos o técnicos [3]. En realidad, **todo desarrollo involucra a personas**: programadores con distintos niveles de confianza, directivos con expectativas comerciales y clientes con necesidades cambiantes [1], [3], [4]. La Tabla VIII sintetiza los actores del ecosistema sociotécnico y lo que la IA puede (y no puede) hacer con cada uno.

La IA **no entiende de dinámicas de equipo**, de temores ante cambios drásticos ni de cómo motivar a un grupo bajo presión de entrega [1], [3]. Si un gestor se limita a seguir ciegamente lo que indica una métrica en pantalla, corre el riesgo de caer en un **liderazgo distante e invisible** que desgasta las relaciones y deteriora el compromiso colectivo [1], [3].

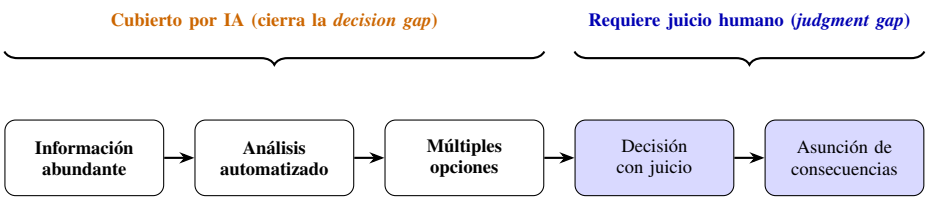


Figura 3: La brecha de juicio: la IA cierra la *decision gap* (información → opciones) pero no la *judgment gap* (opciones → decisión → consecuencias).

Cuadro VIII: El ecosistema social del proyecto de software: actores, necesidades y límites de la IA

Actor	Necesidad principal	Qué puede hacer la IA	Qué no puede hacer la IA
Desarrolladores	Claridad técnica, autonomía, reconocimiento	Resumir especificaciones, detectar deuda técnica	Motivar bajo presión, contener frustraciones
Directivos	Previsibilidad, retorno de inversión	Generar pronósticos, escenarios de costo	Negociar prioridades políticas, sostener compromisos
Clientes	Producto funcional, plazos confiables	Analizar feedback, detectar patrones de uso	Comprender expectativas implícitas, gestionar conflictos
Equipo distribuido	Coordinación, contexto compartido	Traducir idiomas, resumir reuniones	Construir confianza intercultural, mediar en conflictos

La OIT [1] advierte sobre este riesgo: la delegación excesiva en sistemas automatizados puede derivar en decisiones sin autoría clara, donde nadie asume la responsabilidad final. En organizaciones donde la confianza interpersonal es central —como en América Latina—, este riesgo es particularmente sensible [1].

La consecuencia práctica es que el AI Manager debe ejercer una **autoridad visible, explicativa y empática**, capaz de reconstruir la confianza mediante la transparencia y la rendición de cuentas [1]. No se trata de rechazar la métrica, sino de **interpretarla en contexto**: una caída en la velocidad del equipo puede ser una señal de alarma o una consecuencia esperable de una decisión arquitectónica deliberada. Solo el juicio humano puede distinguir entre ambas.

C. Habilidades humanas que ganan valor

En un entorno saturado de respuestas automatizadas, el verdadero aporte diferencial del PM se apoya en **tres capacidades esenciales** [3]. La Tabla IX las sintetiza, contrastando lo que la IA puede hacer con lo que exige juicio humano, y vinculando cada capacidad con evidencia empírica de las fuentes.

C0a. Sense-making: distinguir señal de ruido.: La capacidad de distinguir qué señales importan y cuáles son simple ruido [3]. Frente a decenas de alertas y métricas generadas por el sistema, el líder sabe interpretar qué problemas afectan verdaderamente al negocio y cuáles son secundarios [2], [3]. Esta capacidad **no puede automatizarse fácilmente porque el significado es contextual**: depende de la estrategia, el timing, la cultura y el juicio [3]. Un mismo indicador —por ejemplo, una caída del 15 % en la velocidad del sprint— puede ser una señal de alarma en un proyecto crítico o un efecto esperado de una decisión arquitectónica en otro.

C0b. Coraje estratégico: decidir cuando la información nunca es suficiente.: La determinación para tomar decisiones difíciles cuando la información nunca es 100 % segura [3]. La IA puede listar alternativas y calcular riesgos, pero **decidir pausar una funcionalidad o postergar una fecha límite para cuidar la calidad exige valentía directiva y convicción profesional** [3]. Como sintetiza Nieto-Rodríguez: "la IA puede identificar riesgos; los líderes deben decidir qué riesgos vale la pena correr. La IA puede destacar trade-offs; los líderes deben vivir con las consecuencias" [3]. La tecnología reduce la incertidumbre; **nunca la elimina**.

C0c. Construcción de confianza: el acelerador de ejecución más potente.: El desarrollo de acuerdos genuinos y credibilidad compartida [3]. **Ningún modelo de lenguaje puede inspirar entusiasmo en un equipo estresado** ni convencer a directivos reticentes de apoyar una propuesta técnica compleja [3]. La confianza entre las personas sigue siendo el acelerador de ejecución más potente en cualquier proyecto de software [3]. Y es, quizás, la capacidad más difícil de automatizar: la IA puede analizar el sentimiento de los stakeholders, pero no puede construir una coalición entre intereses contrapuestos ni persuadir a un ejecutivo escéptico [3].

D. Integración: el AI Manager como árbitro del juicio

Las tres capacidades no operan de forma aislada. Se **complementan** en la práctica del AI Manager, que actúa como **árbitro del juicio**: alguien que recibe múltiples señales, las interpreta en contexto, decide con información incompleta y sostiene la confianza del equipo en el proceso [1], [3].

Cuadro IX: Las tres capacidades humanas que ganan valor en la era de la IA

Capacidad	Qué hace la IA	Qué exige el juicio humano	Evidencia empírica
Sense-making (construcción de sentido)	Genera decenas de alertas y métricas	Distinguir qué señales importan y cuáles son ruido; interpretar el contexto estratégico	Nieto-Rodríguez: organizaciones sufren de *signal overload* [3]
Coraje estratégico (*strategic courage*)	Lista alternativas y calcula riesgos	Decidir con información incompleta; asumir consecuencias de trade-offs	Nieto-Rodríguez: "la IA puede generar veinte escenarios, los líderes deben elegir uno" [3]
Construcción de confianza (*trust creation*)	Analiza sentimiento de stakeholders	Inspirar entusiasmo, contener equipos estresados, persuadir a directivos escépticos	Barbosa et al.: "GenAI sirve como aumento, no como reemplazo" (R14) [2]

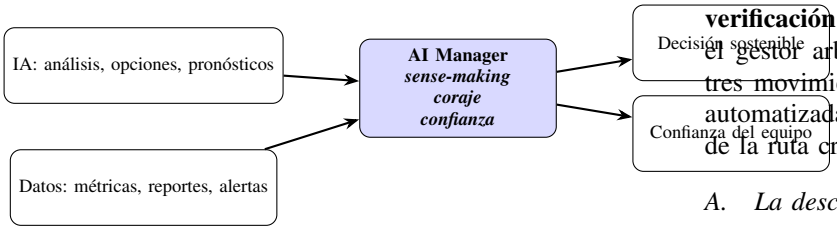


Figura 4: El AI Manager como árbitro del juicio: integra análisis algorítmico y criterio humano para producir decisiones sostenibles y confianza organizacional.

La Figura 4 sintetiza esta integración: la IA provee el análisis y las opciones; el AI Manager aporta el juicio que convierte esas opciones en decisiones sostenibles.

La consecuencia es que el *judgment gap* no se cierra con **más** IA, sino con **mejor** liderazgo. La tecnología es condición necesaria pero no suficiente: sin un gestor capaz de interpretar, decidir y sostener, la abundancia de información se convierte en parálisis. Como sintetiza Nieto-Rodríguez [3], el desafío no es tecnológico sino de liderazgo: *"la IA puede ayudarnos a ver; el liderazgo determina qué hacemos con lo que vemos"*.

V. Gestión de cuellos de botella y verificación de código asistida por IA

En la práctica de los equipos de desarrollo, uno de los mayores desafíos del Project Manager radica en la **falta de visibilidad sobre el avance real del trabajo** [2]. Con frecuencia, el estado reflejado en los tableros de gestión (como ClickUp o Jira) diverge de la realidad técnica: tareas marcadas como completadas carecen de pruebas de integración, o ramas de código permanecen estancadas durante días sin *commits* visibles [1]. Esta desconexión genera cuellos de botella imprevistos y descalibra los plazos de entrega comprometidos [3].

La tesis de esta sección es que **la IA no resuelve el problema de la visibilidad por sí sola**: lo desplaza. Si en la Sección IV se argumentó que la IA acentúa la brecha de juicio, aquí se sostiene que también **acentúa la brecha de verificación**: cuanto más rápido se generan *commits*, más difícil es auditarlos manualmente. La respuesta no es más automatización sin control, sino una **arquitectura de verificación** donde los agentes automatizados auditan, pero el gestor arbitra. Esta sección desarrolla esa arquitectura en tres movimientos: detección temprana de bloqueos, auditoría automatizada sobre el código, y el rol del gestor como árbitro de la ruta crítica.

A. La desconexión entre el tablero y la realidad técnica

El problema de la visibilidad no es un defecto de las herramientas de gestión, sino una **consecuencia estructural** de cómo se reporta el trabajo. Los tableros capturan lo que el equipo *dice* que hace, no lo que el código *realmente* hace. La Tabla X sintetiza las divergencias más frecuentes entre el estado reportado y la realidad técnica, junto con las señales que permiten detectarlas.

La tabla revela un patrón: **el tablero mide actividad, no calidad**. Una tarea puede estar "completada" sin cumplir los criterios de aceptación; un módulo puede estar "integrado" sin respetar el contrato de interfaz. La IA puede cerrar esta brecha porque **no depende de lo que el equipo reporta**, sino de lo que el código evidencia.

B. Detección temprana de bloqueos y estimación predictiva

Tradicionalmente, el seguimiento de tiempos depende de estimaciones empíricas calculadas al inicio del ciclo, las cuales rara vez se cumplen ante imprevistos arquitectónicos [1], [4]. La integración de modelos analíticos permite **anticipar retrasos** cruzando la velocidad histórica del equipo con la actividad del repositorio de versiones [1].

Al monitorear métricas como el **tiempo de apertura de Pull Requests (PR)**, la **frecuencia de integración** y la **tasa de fallos en las canalizaciones de prueba**, la IA actúa como un **sensor preventivo** [1], [2]. Si un módulo crítico presenta una divergencia prolongada respecto al contrato de interfaz pactado, el sistema alerta al gestor **antes de que el bloqueo paralice a otras áreas dependientes** (por ejemplo, impidiendo que el *frontend* deba esperar días a que *backend* entregue un *endpoint* funcional) [2].

La Tabla XI sintetiza las métricas clave, sus umbrales típicos de alerta y la acción que habilita cada una.

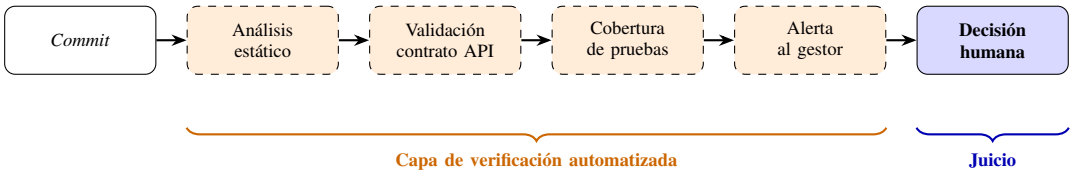
La Figura 5 ilustra el flujo de detección: desde el *commit* hasta la alerta al gestor, con los puntos donde la IA actúa y donde el juicio humano interviene.

Cuadro X: Divergencias entre el estado reportado y la realidad técnica del proyecto

Indicador en tablero	**Realidad técnica posible**	**Señal de alerta**	**Métrica de verificación**
Tarea “completada”	Sin pruebas de integración; sin cobertura	PR sin tests; cobertura < umbral	Cobertura de pruebas por PR
Sprint “en curso”	Ramas estancadas; sin *commits* recientes	PR abierto > N días sin actividad	Tiempo de apertura de PR
Historia “en revisión”	Revisión superficial; deuda técnica acumulada	Comentarios genéricos; *merge* sin validación	Ratio de comentarios por PR
Módulo “integrado”	Contrato de API roto; *endpoints* faltantes	Discrepancia entre esquema OpenAPI y código	Validación automática de contratos

Cuadro XI: Métricas de monitoreo para la detección temprana de bloqueos

Métrica	**Qué mide**	**Umbral de alerta**	**Acción que habilita**
Tiempo de apertura de PR	Días entre apertura y *merge*	> 3 días sin revisión	Destrabar revisión; reasignar revisor
Frecuencia de integración	*Commits* por día por rama	< 1 *commit*/día en rama activa	Verificar bloqueo técnico o desmotivación
Tasa de fallos en CI	% de *pipelines* fallidos	> 20 % en una semana	Pausar *merges*; revisar infraestructura
Divergencia de contrato	Diferencia entre esquema OpenAPI y código	Cualquier discrepancia	Bloquear *merge*; notificar a *backend*
Cobertura de pruebas	% de líneas cubiertas por tests	< 70 % en módulo crítico	Requerir tests antes de *merge*



La IA detecta; el gestor decide qué hacer con la detección

Figura 5: Pipeline de verificación asistida por IA: la automatización cubre la detección, pero la decisión permanece humana.

C. *Auditoría automatizada sobre líneas de código*

Supervisar el avance técnico no exige que el PM examine manualmente cada archivo de la base de código [1]. Mediante **agentes de análisis estático** y modelos integrados al repositorio, la IA puede auditar los cambios en el código (*diffs*) de cada *commit* y contrastarlos con los criterios de éxito especificados en la tarea [2]. La Tabla XII sintetiza los cuatro tipos de auditoría automatizada, su alcance y el riesgo que mitigan.

C0a. Verificación contra criterios de aceptación.: La IA analiza los cambios en el código (*diffs*) de cada *commit* y los contrasta con los criterios de éxito especificados en la tarea. Esto permite comprobar si una funcionalidad fue programada en su totalidad o si solo se maquetó una estructura superficial sin lógica de negocio subyacente [2]. Por ejemplo, si la historia de usuario especifica que un *endpoint* debe manejar errores 401/403/500, la IA verifica que el código incluya esos casos, no solo el *happy path*.

C0b. Cumplimiento de contratos de API.: El sistema valida que los *endpoints* implementados respeten fielmente los esquemas documentados (OpenAPI/Swagger), detectando discrepancias en tipos de datos, códigos de error HTTP o parámetros faltantes **antes de autorizar el *merge*** hacia las ramas compartidas [2], [4]. Esta auditoría es particularmente valiosa en equipos distribuidos, donde el *frontend* y el *backend* pueden estar desarrollándose en paralelo sin comunicación constante.

C0c. Control de calidad y deuda técnica.: Herramientas automatizadas auditan la presencia de malas prácticas, patrones inseguros o falta de cobertura en pruebas unitarias, evitando que la urgencia por cerrar *tickets* degrade la arquitectura general del sistema [1]. La deuda técnica es un riesgo silencioso: no se manifiesta en el tablero hasta que se convierte en un bloqueo. La auditoría automatizada la hace **visible antes de que sea costosa**.

C0d. Ejemplo trabajado: un merge bloqueado por divergencia de contrato.: Supóngase que el equipo de *backend* abre un PR para un *endpoint* de pago. El tablero marca la tarea como “completada”. Sin embargo, el agente de auditoría detecta que:

- El esquema OpenAPI especifica un campo `currency` de tipo `string` con valores `ARS`, `USD`, `EUR`.

Cuadro XII: Tipos de auditoría automatizada sobre el código

Tipo de auditoría	Qué verifica	Cómo lo hace	Riesgo que mitiga
Contra criterios de aceptación	Que la funcionalidad esté completa, no solo maquetada	Compara *diff* con criterios de la historia de usuario	Funcionalidad incompleta
Contratos de API	Que los *endpoints* respeten el esquema OpenAPI/Swagger	Valida tipos de datos, códigos HTTP, parámetros	Integración rota
Calidad y deuda técnica	Malas prácticas, patrones inseguros, falta de cobertura	Análisis estático + métricas de cobertura	Degradación arquitectónica
Seguridad	Vulnerabilidades conocidas, secretos expuestos	Escaneo de dependencias y análisis de *diffs*	Incidentes de seguridad

- El código implementa `currency` como `integer` (código numérico ISO).
- El *endpoint* no maneja el error 403 (pago rechazado por falta de fondos).

El sistema **bloquea el *merge*** y alerta al gestor. El PM no necesita revisar el código línea por línea: recibe un reporte sintético que dice "PR #1234: divergencia en campo `currency`; falta manejo de error 403". Con esa información, negocia con el equipo de *backend* la corrección antes de integrar. **El tablero decía "completado"; la auditoría dijo "no cumple".** La IA hizo visible la brecha.

D. El gestor como árbitro de la ruta crítica

La automatización de la auditoría técnica **no excluye la intervención del gestor**; por el contrario, **transforma su función operativa** [1], [3]. Al recibir reportes sintéticos generados por la IA sobre qué PRs cumplen las especificaciones y cuáles introducen riesgos, el PM puede **negociar prioridades con fundamentos concretos** [2], [3].

En lugar de consumir tiempo en discusiones informales sobre el porcentaje de avance, el liderazgo se orienta a:

1. **Destrabar dependencias** entre integrantes (por ejemplo, priorizar la revisión de un PR que bloquea a tres módulos dependientes).
2. **Ajustar el alcance** cuando los tiempos de compilación e integración se desvían (por ejemplo, postergar una funcionalidad no crítica para no comprometer la fecha de entrega).
3. **Garantizar que el producto cumpla los estándares** requeridos antes de su despliegue en entornos productivos [1], [3].

La Figura 6 sintetiza este rol: la IA provee reportes sintéticos; el gestor arbitra entre velocidad, calidad y alcance.

La consecuencia es que el *judgment gap* de la Sección IV se manifiesta aquí en su forma más concreta: **la IA puede decir qué PR está bloqueado, pero no puede decidir si conviene destrabarlo, postergarlo o cancelarlo**. Esa decisión depende del contexto estratégico, de la presión de los *stakeholders* y de la tolerancia al riesgo del equipo. La automatización hace visible el problema; **el juicio humano decide qué hacer con esa visibilidad**.

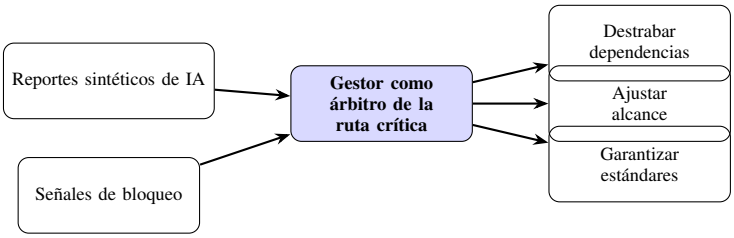


Figura 6: El gestor como árbitro de la ruta crítica: la IA provee reportes, el gestor decide cómo actuar sobre ellos.

VI. Tensiones y brechas

A lo largo de este artículo se argumentó que la IA transforma el rol del PM, pero no lo sustituye. Las secciones anteriores describieron esa transformación en términos de desplazamiento: del papeleo a la ingeniería de contexto (Sección II), del control estático al control adaptativo (Sección III), de la analítica al juicio (Sección IV), y de la supervisión manual a la verificación automatizada (Sección V). Sin embargo, esta transformación no es armónica. Genera **tensiones estructurales** que la literatura entusiasta suele pasar por alto y que este artículo busca explicitar.

La tesis de esta sección es que **el AI Manager no resuelve las tensiones de la gestión tradicional: las redistribuye**. Algunas se atenúan (la carga administrativa, la falta de visibilidad), pero otras se amplifican (la brecha de criterio, el riesgo de autoría difusa, la dependencia de marcos normativos que no siempre existen en el contexto local). Identificar estas tensiones es condición necesaria para un liderazgo responsable. La Tabla XIII las sintetiza.

A. La paradoja senior/junior: velocidad sin criterio, criterio sin velocidad

La primera tensión es la que ya se anticipó en la Sección II: los gestores *seniors* tienen el juicio pero desconfían de la herramienta; los *juniors* tienen la soltura pero carecen del criterio para detectar errores [2]. Esta asimetría no es un problema menor: es una **falla estructural en la distribución de capacidades**.

Barbosa et al. [2] documentan que los *seniors* priorizan métodos tradicionales probados y prestan mayor atención a la seguridad y la gobernanza, mientras que los *juniors* adoptan la IA con naturalidad y confían rápidamente en sus respuestas. Un entrevistado *senior* lo admite con crudeza:

> *"Como persona senior, puede que no me sienta cómodo usando GenAI y eso podría convertirse en un problema. Alguien que llegue como junior y*

Cuadro XIII: Tensiones estructurales de la transición hacia el AI Manager

Tensión	Qué se gana	Qué se pierde (o se arriesga)	Fuente
Velocidad vs. criterio	Eficiencia administrativa (70 %)	Criterio para detectar errores sutiles	Barbosa et al. [2]
Automatización vs. autoría	Decisiones rápidas	Claridad sobre quién responde por ellas	OIT, en López Hurtado [1]
Marcos globales vs. capacidades locales	Estándares de gobernanza	Implementación efectiva en LATAM	CEPAL, en López Hurtado [1]
Eficiencia vs. deuda técnica	Cierre rápido de *tickets*	Degradación arquitectónica silenciosa	Sommerville [4]

> *sea muy práctico con GenAI podría darme mejores resultados."* (R14) [2]

La consecuencia práctica es que **ninguno de los dos perfiles, por separado, es suficiente**. El senior que no adopta la IA pierde eficiencia; el junior que confía ciegamente en ella pierde criterio. La organización que no logra integrar ambas miradas queda atrapada en una de dos trampas: **lentitud sin innovación** (seniors solos) o **velocidad sin solidez** (juniors solos). La superación de esta tensión no es técnica sino organizacional: exige diseñar **equipos híbridos** donde el junior acelera la adopción y el senior valida los entregables [1], [2].

B. Automatización sin autoría: el riesgo del liderazgo invisible

La segunda tensión es la que la OIT [1] denomina **liderazgo invisible**: la delegación excesiva en sistemas automatizados puede derivar en decisiones sin autoría clara, donde nadie asume la responsabilidad final. Este riesgo es particularmente sensible en organizaciones donde la confianza interpersonal es un componente central de la cultura laboral, como en América Latina [1].

El problema no es que la IA decida, sino que **el gestor deje de decidir sin que nadie lo note**. Si un tablero recomienda reasignar recursos y el gestor acepta sin cuestionar, ¿quién responde si el proyecto fracasa? La IA no puede asumir consecuencias; el gestor que delegó tampoco puede excusarse en el algoritmo. La tensión es, en el fondo, una **crisis de autoría**: la automatización hace que las decisiones sean más rápidas, pero también más difusas en cuanto a responsabilidad [1], [3].

La superación de esta tensión exige lo que López Hurtado [1] llama una **autoridad visible, explicativa y empática**: el AI Manager debe poder explicar por qué tomó cada decisión, qué información consideró y qué margen de error asumió. La transparencia no es un adorno ético, sino una **condición de legitimidad** del liderazgo aumentado.

C. La brecha normativa: marcos globales, capacidades locales

La tercera tensión es de naturaleza institucional. Los marcos de gobernanza algorítmica —el EU AI Act, el NIST AI RMF— establecen estándares exigentes de auditoría, explicabilidad y supervisión humana [1]. Pero estos marcos fueron diseñados en contextos con infraestructura, talento y capacidades institucionales que no siempre existen en América Latina. La CEPAL [1] advierte que la región enfrenta una combinación de desafíos estructurales —infraestructura desigual, limitada disponibilidad de talento especializado— que dificulta la adopción sistemática de la IA en las organizaciones.

La tensión es la siguiente: **los estándares globales son necesarios pero insuficientes**. Necesarios, porque sin ellos la IA se convierte en una caja negra sin control. Insuficientes, porque su implementación efectiva requiere capacidades que la región no siempre tiene. El riesgo es que la brecha normativa se convierta en una **brecha de gobernanza**: países con marcos robustos pero sin capacidad de implementación, o países con capacidad pero sin marcos.

La superación de esta tensión no es técnica sino política. Requiere, como señalan la CEPAL y la CAF [1], políticas públicas de alfabetización algorítmica, inversión en infraestructura de datos y fortalecimiento de la cooperación intersectorial. El AI Manager latinoamericano enfrenta una tarea doble: **promover la adopción tecnológica y proteger los valores sociales**, garantizando que la innovación sirva al desarrollo humano y no lo subordine [1].

D. Velocidad vs. deuda técnica: la trampa de la eficiencia

La cuarta tensión es la que se anticipó en la Sección V: la IA acelera el cierre de *tickets*, pero también puede acelerar la acumulación de deuda técnica si no hay auditoría automatizada [4]. La urgencia por cerrar tareas puede llevar a *merges* sin tests, a la maquetación superficial de funcionalidades o a la integración de código que no respeta los contratos de interfaz.

La tensión es particularmente insidiosa porque **no se manifiesta en el tablero hasta que es costosa**. Un proyecto puede mostrar un 90 % de avance y, sin embargo, tener una arquitectura frágil que colapsará en producción. La IA puede hacer visible esa fragilidad —mediante auditoría automatizada de *diffs*, validación de contratos y análisis de cobertura—, pero **solo si el gestor decide usarla para eso** y no solo para acelerar el cierre de *tickets* [1], [2].

La superación de esta tensión exige lo que en la Sección V se denominó **arquitectura de verificación**: un pipeline donde la IA audita, pero el gestor arbitra entre velocidad, calidad y alcance. La eficiencia sin verificación no es eficiencia: es **deuda técnica diferida**.

E. Síntesis: el AI Manager como gestor de tensiones

Las cuatro tensiones no se resuelven de una vez y para siempre. Se **gestionan**. El AI Manager no es el gestor que elimina las tensiones, sino el que las **habita con criterio**: acepta la velocidad de la IA sin renunciar al juicio humano, automatiza la detección sin delegar la decisión, adopta marcos globales sin ignorar las capacidades locales, y acelera la entrega sin sacrificar la arquitectura.

Esta es, quizás, la diferencia más profunda entre el PM tradicional y el AI Manager. El primero gestiona recursos; el segundo **gestiona tensiones**. Y las gestiona no con más herramientas, sino con **mejor juicio**.

VII. Conclusiones

Este artículo examinó la transformación del Project Manager en la era de la IA a través de cuatro desplazamientos: del papeleo a la ingeniería de contexto, del control estático al control adaptativo, de la analítica al juicio, y de la supervisión manual a la verificación automatizada. La conclusión central es que **la IA no reemplaza al gestor: desplaza su valor**. El PM deja de ser un administrador de tareas para convertirse en un **habilitador estratégico y un AI Manager** [1], [2].

A. Los cuatro desplazamientos

A0a. Del papeleo a la ingeniería de contexto.: La primera conclusión es que la automatización de tareas administrativas **libera tiempo pero no resuelve el problema de fondo**. La IA puede resumir reuniones, redactar minutas y corregir correos, pero si el gestor no sabe qué hacer con el tiempo liberado, la ganancia se diluye. La habilidad que marca la diferencia es la **ingeniería de contexto**: la capacidad de alimentar a la IA con reglas de negocio, contratos de interfaz y restricciones arquitectónicas para que los *outputs* sean aplicables y no genéricos [2]. Como sintetiza uno de los entrevistados por Barbosa et al., el *prompt engineering* es darle una tarea a un *minion*; el contexto es usarlo como copiloto [2].

A0b. Del control estático al control adaptativo.: La segunda conclusión es que la IA transforma el ciclo de vida del software: la planificación deja de ser un documento estático para convertirse en un proceso vivo, y el control deja de ser una auditoría sancionatoria para convertirse en un seguimiento en tiempo real [1]. Pero esta transformación exige un perfil híbrido: el AI Manager debe dominar tres dimensiones —técnica, cognitiva y ética— que no operan de forma jerárquica sino complementaria [1]. Sin dimensión técnica, el gestor no comprende los límites de los modelos; sin dimensión cognitiva, confunde probabilidad con certeza; sin dimensión ética, automatiza sin dirección.

A0c. De la analítica al juicio.: La tercera conclusión es la más contraintuitiva: **la IA acentúa la brecha de juicio en lugar de cerrarla**. La tecnología cierra la *decision gap* —la distancia entre reconocer que se necesita una decisión y contar con el análisis para prepararla—, pero amplifica la *judgment gap* —la distancia entre tener el análisis y saber qué hacer con él [3]. Un tablero puede recomendar recortar funcionalidades o reasignar módulos, pero no asume las consecuencias si el cliente queda disconforme o si el producto falla en producción. Las consecuencias siempre las asumen las personas [3]. Por eso, las tres capacidades que ganan valor —*sense-making*, coraje estratégico y construcción de confianza— son precisamente las que no pueden automatizarse [3].

A0d. De la supervisión manual a la verificación automatizada.: La cuarta conclusión es que la IA no elimina la necesidad de supervisión: la **redistribuye**. Los agentes automatizados pueden auditar *diffs*, validar contratos de API y detectar deuda técnica antes de que se consolide [1], [2]. Pero esa automatización solo es valiosa si el gestor la usa para **arbitrar la ruta crítica**, no para acelerar el cierre de *tickets*. La diferencia entre eficiencia y deuda técnica diferida está en la arquitectura de verificación: la IA detecta, el gestor decide.

B. Las tensiones persistentes

La segunda conclusión es que **la transformación no es armónica**. Genera tensiones estructurales que la literatura entusiasta suele pasar por alto: la paradoja senior/junior (velocidad sin criterio, criterio sin velocidad), el riesgo de liderazgo invisible (automatización sin autoría), la brecha normativa (marcos globales, capacidades locales) y la trampa de la eficiencia (velocidad vs. deuda técnica) [1]–[3].

Estas tensiones no se resuelven de una vez y para siempre. Se **gestionan**. El AI Manager no es el gestor que las elimina, sino el que las habita con criterio. Y las gestiona no con más herramientas, sino con **mejor juicio**. Esta es, quizás, la diferencia más profunda entre el PM tradicional y el AI Manager: el primero gestiona recursos; el segundo **gestiona tensiones**.

C. Implicaciones prácticas

La tercera conclusión es de naturaleza práctica. Las organizaciones que aspiren a evolucionar hacia estructuras de gestión inteligentes deberán:

1. **Fortalecer la formación de sus líderes** en analítica de datos, pensamiento sistémico y ética aplicada [1].
2. **Crear mecanismos institucionales de gobernanza algorítmica** que garanticen la transparencia y la rendición de cuentas [1].
3. **Diseñar equipos híbridos** donde el junior acelera la adopción y el senior valida los entregables [2].
4. **Construir una cultura de confianza y aprendizaje continuo**, donde el error se entienda como oportunidad de mejora y los datos se conciban como bien común [1].

En el contexto latinoamericano, estas implicaciones adquieren matices específicos. La región enfrenta una combinación de desafíos estructurales —infraestructura desigual, limitada disponibilidad de talento especializado— que dificulta la adopción sistemática de la IA [1]. Pero también posee un potencial singular para liderar modelos de innovación inclusiva, basados en el principio de *IA con propósito público* [1]. El AI Manager latinoamericano enfrenta una tarea doble: **promover la adopción tecnológica y proteger los valores**

sociales, garantizando que la innovación sirva al desarrollo humano y no lo subordine.

D. Limitaciones y agenda de investigación

Este artículo tiene limitaciones que conviene explicitar. Primero, se apoya en tres estudios recientes que, si bien son representativos de la literatura emergente, no agotan el campo. Segundo, el análisis es fundamentalmente conceptual: no incluye evidencia empírica primaria sobre la implementación del AI Manager en organizaciones latinoamericanas. Tercero, las tensiones identificadas en la Sección VI son hipótesis derivadas de la literatura, no hallazgos validados empíricamente en el contexto regional.

La agenda de investigación que se desprende es clara:

- **Estudios de caso** en sectores estratégicos de América Latina (educación, agroindustria, energía, salud) para evaluar empíricamente el desempeño del AI Manager [1].
- **Indicadores de madurez ética y algorítmica** que permitan medir la capacidad institucional para integrar la IA de forma responsable [1].
- **Análisis longitudinales** que capturen la evolución de la brecha senior/junior a medida que los *juniors* ganan experiencia y los *seniors* se adaptan [2].
- **Estudios comparados** entre marcos normativos (EU AI Act, NIST) y capacidades locales de implementación en LATAM [1].

E. Reflexión final

La transformación del Project Manager en la era de la IA no es, en el fondo, una historia sobre tecnología. Es una historia sobre **qué significa liderar cuando la información deja de ser escasa**. Durante más de un siglo, la gestión buscó resolver el problema de la escasez de información; la IA está resolviendo ese problema más rápido de lo que esperábamos. El próximo desafío es distinto: **cómo liderar cuando la información es abundante, cómo priorizar cuando las opciones se multiplican, cómo ejercer el juicio cuando las máquinas pueden generar recomendaciones infinitas** [3].

Estas no son preguntas tecnológicas. Son preguntas de liderazgo. Y pueden convertirse en las preguntas definitorias de la próxima década. Las organizaciones que triunfen no serán necesariamente las que tengan la IA más avanzada, sino las que combinen la inteligencia artificial con algo mucho más difícil de escalar: **el juicio humano** [3].

Porque los proyectos rara vez fracasan porque falte información. Fracasan porque las personas eligen las prioridades equivocadas, postergan decisiones difíciles, evitan *trade-offs* incómodos o malinterpretan una realidad cambiante [3]. La IA puede ayudarnos a ver. **El liderazgo determina qué hacemos con lo que vemos**. Y en la Economía de Proyectos, esa diferencia determina cada vez más quién crea el futuro y quién reacciona a él.

REFERENCIAS

[1] J. López Hurtado, "Del Project Manager al AI Manager: Transformación de los roles gerenciales en la era digital," *Innovación y Gerencia. Revista científica arbitrada*, vol. 12, no. Edición Especial 1, pp. 43–52, Jun. 2026.

[2] B. D. Barbosa, C. Trocin, and B. Palese, "From potential to practice: The use of GenAI in project management," in *Proceedings of the International Research Workshop on IT Project Management (IRWITPM)*, ser. AIS Electronic Library (AISeL), Dec. 2025. [Online]. Available: https://aisel.aisnet.org/irwitpm2025

[3] A. Nieto-Rodriguez, "Why AI makes project leadership more human, not less: Reimagining project management for a new era," *PM World Journal*, vol. 15, no. 7, pp. 1–10, Jul. 2026. [Online]. Available: https://pmworldlibrary.net/

[4] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education Limited, 2016.